

OPTI DECISION SUITE

Documentación

Integrantes:

- Noelia Sejas
- Lucas David Loayza
- Favio Valenzuela
- Nathan Ezequías Torrico Torrez

Docente: Myrna Gloria Baena Paniagua

Cochabamba, Noviembre 2026

Índice

CAPÍTULO I: INTRODUCCIÓN....1
1.1. Antecedentes....1
1.2. Objetivos....1
1.2.1. Objetivo General....1
1.2.2. Objetivos Específicos....1
1.3. Justificación....1
CAPÍTULO II: MARCO TEÓRICO....2
2.2. Métodos para la Resolución de Ecuaciones No Lineales....2
2.2.1. Método de Punto Fijo....2
2.2.2. Método de Bisección....2
2.2.3. Método de Regula Falsi (Posición Falsa)....3
2.2.4. Método de la Secante....3
2.2.5. Método de Newton-Raphson (Primer Orden)....4
2.2.6. Método de Newton-Raphson (Segundo Orden)....4
2.3. Métodos para la Resolución de Sistemas de Ecuaciones Lineales (A * x = b)....5
2.3.1. Método de Eliminación Gaussiana con Pivotaje Parcial....5
2.3.2. Método de Gauss-Jordan con Pivotaje Parcial....5
2.3.3. Método Iterativo de Jacobi....6
2.3.4. Método Iterativo de Gauss-Seidel....6
CAPÍTULO III: INGENIERÍA DEL PROYECTO....8
3.1. Arquitectura....8
3.2. Requerimientos Funcionales....8
3.2. Requerimientos No Funcionales....8
CAPÍTULO IV: INGENIERÍA DE DESARROLLO....10
4.1. Diagrama de caso de usos general....10

CAPÍTULO I: INTRODUCCIÓN

1.1. Antecedentes

1.2. Objetivos

1.2.1. Objetivo General

Desarrollar una plataforma prototipo de optimización de compras y análisis analítico para el sector ferretero en Cochabamba, para compradores implementando la búsqueda multicriterio y ferreteros mediante la gestión de inventarios y retención.

1.2.2. Objetivos Específicos

- Especificar, diseñar, dirigir y revisar el desarrollo de la plataforma usando SPEC-DRIVEN DEVELOPMENT (Desarrollo dirigido por especificaciones)
- Implementar el código dado por la IA pieza por pieza para cada módulo.
- Probar el código dado aplicando el filtro de criterio:
    - ¿Qué pasa si llega un dato inesperado?
    - ¿Hay algo que comprometa la seguridad?
    - ¿Podría explicar este código a otra persona?
- Construir una interfaz gráfica de usuario con diseño adaptable y reportes tabulares detallados.

1.2.3. Alcance V1

Esta plataforma prototipo tiene un alcance academico, solo para cómo se aplicaría los métodos de la Investigación Operativa II en una situación real, asimismo esta plataforma tendrá las siguientes funcionalidades.

- Registro personal de usuarios mediante Google OAuth.
- Vista exclusiva para administradores/ferreteros para carga masiva y actualización de catálogo, precios y stock en tiempo real.

- Buscador geoespacial y ordenamiento AHP (Precio, Distancia, Reputación, Stock).
- Gestión de lotes óptimos con modelo EOQ y Punto de Reorden (ROP).
- Evaluación de alternativas de compra bajo incertidumbre mediante Árboles de Decisión (VME y VEIP).
- Módulo de Predicción para la simulación de comportamiento de navegación y conversión de compradores mediante Cadenas de Markov con Estados Absorbentes.

1.2.4. Límites

- Procesamiento de pagos con tarjeta o pasarelas bancarias reales.
- Geolocalización por GPS en tiempo real (se usan coordenadas estáticas fijas).

1.2.5. Tecnología del proyecto

- **Backend:** Python 3.12 con FastAPI
- **Librerías Matemáticas (IO2):** NumPy (álgebra lineal, matrices AHP y Markov) y SciPy
- **Base de Datos:** PostgreSQL 16 con extensión PostGIS (almacenamiento y consultas geoespaciales ST_DWithin / ST_DistanceSphere)
- **Frontend:** HTML5, CSS3, JavaScript Vanilla (Fetch API para consumo de endpoints)
- **Mapas e Interfaz Geoespacial:** Leaflet.js + OpenStreetMap (Renderizado de mapa de Bolivia y geolocalización)
- **Autenticación:** OAuth2 / Google Sign-In + JWT (JSON Web Tokens)

1.3. Justificación

- **Justificación Tecnológica**

Uso de Python, NumPy para álgebra matricial de alto rendimiento y SymPy para derivación simbólica automática.

- **Justificación Académica**

Proporciona un entorno interactivo que ayuda al estudiante a verificar la convergencia iterativa y el comportamiento de los costos y reglas de negocio además de reducir el proceso moroso de cálculo a mano.

CAPÍTULO II: MARCO TEÓRICO

2.2. Métodos para la Resolución de Ecuaciones No Lineales

2.2.1. Método de Punto Fijo

El método de punto fijo es un procedimiento iterativo que permite hallar raíces transformando la función original igualada a cero en su forma equivalente despejada (la función g(x), como vimos en clase).

- **Datos de entrada:** Función despejada g(x), valor inicial, tolerancia de error (E) y límite máximo de iteraciones.

Algoritmo paso a paso:

1. Establecer el valor inicial y la tolerancia de error.
2. Evaluar la función despejada para obtener el siguiente valor usando la fórmula del punto fijo.
3. Calcular el error absoluto entre la iteración nueva y la anterior.
4. Verificar el criterio de parada: Si el error es menor o igual a E, finalizar y declarar la raíz. Si no, actualizar el valor y volver al paso 2.

2.2.2. Método de Bisección

Método cerrado que divide a la mitad un intervalo donde se garantiza que hay al menos una raíz por cambio de signo.

- **Datos de entrada:** Función, extremo inferior del intervalo, extremo superior, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Calcular la estimación teórica del número de iteraciones necesarias (la fórmula del logaritmo natural visto en tus apuntes).
2. Calcular el punto medio del intervalo (fórmula del punto medio).
3. Evaluar la función en el punto medio y en el extremo inferior.
4. Evaluar el criterio de cambio de signo para redefinir el intervalo: si la multiplicación de las funciones es negativa, la raíz está en el subintervalo izquierdo; si es positiva, está en el derecho.
5. Calcular el error actual basado en el ancho del intervalo (o evaluando la función en el punto medio).
6. Si el error es menor o igual a E, finalizar y tomar el punto medio como raíz aproximada. De lo contrario, repetir.

2.2.3. Método de Regula Falsi (Posición Falsa)

Algoritmo de intervalo que une los puntos extremos mediante una línea recta secante para aproximar la posición de la raíz en su intersección con el eje horizontal.

- **Datos de entrada:** Función, punto inicial menor, punto inicial mayor, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Evaluar la función en los extremos del intervalo.
2. Calcular el punto de aproximación usando la fórmula de la recta secante (Regula Falsi).
3. Evaluar la función en ese punto y determinar el cambio de signo con relación a los extremos para actualizar el intervalo.
4. Calcular el error de la iteración (diferencia entre el punto nuevo y el anterior).
5. Si el error es menor o igual a E, concluir y retornar el punto. Si no, continuar iterando.

2.2.4. Método de la Secante

Método abierto que aproxima la derivada de la función mediante la pendiente de la recta secante formada por dos puntos iterativos previos.

- **Datos de entrada:** Función, primer punto inicial, segundo punto inicial, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Evaluar la función en los dos puntos iniciales.
2. Calcular el siguiente valor utilizando la fórmula del método de la secante.
3. Calcular el error absoluto actual.
4. Si el error es menor o igual a E, finalizar y tomar el valor como solución. Si no, actualizar las variables para la siguiente iteración y volver al paso 1.

2.2.5. Método de Newton-Raphson (Primer Orden)

Método abierto que utiliza la primera derivada de la función para trazar rectas tangentes en cada iteración y aproximar rápidamente la raíz.

- **Datos de entrada:** Función, primera derivada calculada, valor inicial, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Evaluar la función y su primera derivada en el punto actual.
2. Verificar que la derivada sea diferente de cero para evitar división entre cero.
3. Calcular el siguiente punto iterativo mediante la fórmula de Newton-Raphson de primer orden.
4. Calcular el error. Si es menor o igual a E, detener el algoritmo. De lo contrario, actualizar el valor y repetir.

2.2.6. Método de Newton-Raphson (Segundo Orden)

Variante de aceleración que incorpora la segunda derivada de la función para lograr una mayor velocidad de convergencia.

- **Datos de entrada:** Función, primera derivada, segunda derivada, valor inicial, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Evaluar la función, la primera y la segunda derivada en el valor actual.
2. Calcular el denominador de la fórmula.
3. Verificar que el denominador sea diferente de cero.
4. Calcular el nuevo punto con la fórmula de segundo orden.
5. Evaluar el criterio de parada calculando el error.

2.3. Métodos para la Resolución de Sistemas de Ecuaciones Lineales (A * x = b)

2.3.1. Método de Eliminación Gaussiana con Pivotaje Parcial

Método directo que transforma el sistema original en uno equivalente con matriz triangular superior mediante operaciones elementales de fila y pivoteo por columna.

- **Datos de entrada:** Matriz de coeficientes A, vector de términos independientes b.

Algoritmo paso a paso:

1. Construir la matriz aumentada del sistema.
2. Para cada columna: seleccionar como elemento pivote el valor de mayor valor absoluto en la columna; intercambiar la fila actual con la del pivote.
3. Para las filas de abajo: calcular el multiplicador (división de elementos) y restar a la fila el producto del multiplicador por la fila pivote.
4. Realizar la sustitución hacia atrás para despejar sucesivamente los componentes del vector solución, empezando desde la última variable hasta la primera.
5. Retornar el vector solución.

2.3.2. Método de Gauss-Jordan con Pivotaje Parcial

Método directo que reduce la matriz aumentada del sistema a una matriz identidad, obteniendo la solución directamente en el vector resultante.

- **Datos de entrada:** Matriz de coeficientes A, vector de términos independientes b.

Algoritmo paso a paso:

1. Formar la matriz aumentada.
2. Para cada columna: realizar pivotaje parcial buscando el valor absoluto máximo en la columna e intercambiando filas.
3. Normalizar la fila pivote dividiendo todos sus elementos entre el pivote.
4. Para todas las demás filas (arriba y abajo), restar la fila pivote multiplicada por el elemento correspondiente, logrando ceros por encima y por debajo del pivote.
5. Extraer el vector solución directamente de la última columna de la matriz reducida.

2.3.3. Método Iterativo de Jacobi

Método iterativo que resuelve el sistema despejando cada variable y utilizando exclusivamente los valores del vector de la iteración anterior para calcular la nueva iteración.

- **Datos de entrada:** Matriz A, vector b, vector inicial de ceros, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Establecer la tolerancia de error.
2. Despejar cada variable de su ecuación en función de las demás.
3. Definir el vector de aproximación inicial.
4. En cada iteración, calcular de manera independiente cada variable utilizando los valores de la iteración previa.
5. Calcular la norma vectorial del error entre iteraciones (la raíz cuadrada de la suma de las diferencias al cuadrado).

6. Si la norma es menor o igual a E, finalizar y retornar el vector. Si no, incrementar la iteración y volver al paso 4.

2.3.4. Método Iterativo de Gauss-Seidel

Método iterativo que acelera la convergencia sustituyendo de forma inmediata los valores de las variables que se van calculando dentro de la misma iteración.

- **Datos de entrada:** Matriz A, vector b, vector inicial de ceros, tolerancia de error y límite de iteraciones.

Algoritmo paso a paso:

1. Fijar la tolerancia de error.
2. Despejar secuencialmente cada variable de cada ecuación del sistema.
3. Inicializar el vector de trabajo.
4. Para cada componente en la iteración actual: calcular el nuevo valor e insertar de inmediato dicho resultado en el vector para los cálculos de las siguientes componentes dentro del mismo ciclo (sustitución recursiva).
5. Evaluar la diferencia de error individual por variable o mediante el residuo de la ecuación.
6. Si todos los errores por componente o la norma general cumplen la condición, concluir. En caso contrario, proceder con la siguiente iteración.

CAPÍTULO III: INGENIERÍA DEL PROYECTO

3.1. Arquitectura

La arquitectura del proyecto será una arquitectura vertical por dominio / Módulos de I.O. (Feature-First / Vertical Slices)

```
app/
├── modules/
│   ├── autenticacion/  # OAuth Google, JWT, usuarios
│   ├── buscador_ahp/   # Buscador Leaflet, PostGIS, algoritmo AHP
│   ├── inventarios_eoq/# EOQ, EPQ, Faltantes, Descuentos, ROP
│   ├── decisiones/     # Incertidumbre (Wald, Savage...) y Riesgo (VME/VEIP)
│   ├── markov/     # Matrices de conversión y estados absorbentes
│   └── colas_mm1/  # Tiempos de espera y tasas lambda/mu
├── shared/         # Conexión BD, utilidades comunes de NumPy
└── main.py
```

Donde cada método (o tema del temario) tiene su propia carpeta dedicada, y dentro de ella habitan los archivos necesarios para hacer funcionar ese método de forma independiente.

```
app/modules/markov/
├── __init__.py        # Indica que es un paquete de Python
├── routes.py          # Define las rutas/endpoints (ej. POST /api/markov/simular)
├── engine.py  # Código matemático puro
|              👉# (NumPy, cálculo de N = (I-Q)^-1 y B = N*R)
```

```
└── schemas.py  # Validación de los datos JSON de entrada y
salida
```

En lugar de tener un solo archivo gigante como app.py o carpetas gigantescas globales, cada carpeta de módulo funciona como una mini-aplicación autónoma.

3.2. Requerimientos Funcionales

- RF-01

  Permite al usuario ingresar matrices A y vectores b dinámicamente.

- RF-02

  Opción para seleccionar el criterio de evaluación de todos los tipos de error

- RF-03

  Derivación simbólica automática para métodos de Newton-Raphson

- RF-04

  Debe tener un modal para ver geogebra y poder graficar las función o ecuaciones dadas.

  - Subconscientemente debe poder exportar su gráfica si así lo desea el estudiante.

- RF - 05

  Se debe poder exportar en PDF los cálculos hechos

- RF - 06

  Tres menús de Errores

- RF - 07

  Debe contener un selector de métodos

- RF - 08

3.2. Requerimientos No Funcionales

- El tiempo de cálculo debe ser menor a 20 segundos.
- Todos los métodos integrados en la aplicación deben haber sido enseñados por el docente de la materia.
- La interfaz debe tener un aprobado dentro el cuestionario SUS de Jhon Broke

CAPÍTULO IV: INGENIERÍA DE DESARROLLO

4.1. Diagrama de caso de usos general